

MODULE 1: INTRODUCTION

Contents

1. Drawbacks of Classical Waterfall Model	1
2. Requirements Analysis and Specification	1
3. Metrics for Software Project Size Estimation.....	3
4. Function Point Metrics	4
5. COCOMO Model (Constructive Cost Model).....	6
6. COCOMO II (Constructive Cost Model II)	10
7. Cost-Benefit Analysis (CBA)	11
8. Characteristics of a Good SRS Document.....	12
9. Advantages & disadvantages of the SDLC models	12

Drawbacks of Classical Waterfall Model

1. **Inflexibility and Rigidity:** Once a phase is completed, it's difficult to go back if changes are required, making the model rigid. Each phase must be fully completed before moving to the next, leaving no room for overlap or iterative feedback.
2. **Late Discovery of Issues:** Testing begins only after development is complete, leading to late identification of issues, which can be costly to fix. Early risk mitigation is not possible since problems are only discovered in the testing phase.
3. **Poor Accommodation of Changes:** The model struggles with evolving requirements, as it assumes all requirements are known upfront. Any changes later in the process result in high costs, often requiring redesign, recoding, and retesting.
4. **High Risk of Project Failure:** Feedback is delayed until late stages, increasing the risk that the final product may not meet user expectations. Customer feedback arrives late, which can result in misaligned deliverables.
5. **Inefficient for Complex Projects:** Waterfall's linear structure is unsuitable for complex or large projects that require flexibility. It doesn't support prototypes or mock-ups to validate early assumptions.
6. **Poor Visibility and Monitoring:** Progress is hard to measure until the coding phase, making it difficult for management to gauge project status. There are no interim deliverables to provide checkpoints.
7. **Unsuitable for Long-Term Maintenance:** Maintaining and updating the system is challenging because the model doesn't support iterative refinements, and outdated documentation makes future maintenance difficult.

Requirements Analysis and Specification

Requirements analysis and specification are critical phases in the **Software Development Life Cycle (SDLC)** that aim to thoroughly understand, document, and communicate the functionality and constraints of a system. These phases ensure that developers, stakeholders, and end-users have a clear, shared understanding of what the system should achieve before design and development begin.

A. Requirements Analysis: Requirements analysis is the process of identifying, gathering, and defining the **needs and expectations** of the stakeholders for a new or modified system. The goal is to understand what the system should do (functional requirements) and the constraints or conditions under which it must operate (non-functional requirements). Key Steps in Requirements Analysis:

1. **Stakeholder Identification:** Identify the people who have a direct or indirect interest in the project (e.g., users, customers, management, regulatory authorities).
2. **Requirements Elicitation:** Gather requirements through interviews, surveys, workshops, observation, and analysis of existing systems.
3. **Categorizing Requirements:**
 - a. **Functional Requirements:** Define specific behaviours or functions the system must perform (e.g., "The system must allow users to log in using a username and password").
 - b. **Non-functional Requirements:** Define constraints on the system, such as performance, security, usability, reliability, or scalability (e.g., "The system must handle 10,000 concurrent users").
4. **Conflict Resolution:** Identifying and resolving conflicts between different stakeholders' needs or requirements.
5. **Feasibility Study:** Analysing the technical and business feasibility of the requirements to ensure that they can be implemented within constraints like time, budget, and resources.

B. Requirements Specification: The **requirements specification** is the process of documenting the results of the analysis in a clear, structured format. It formalizes the requirements so that they can be clearly communicated to all stakeholders, including designers, developers, and testers. The final document is often called a Software Requirements Specification (SRS). Key Components of Requirements Specification:

Introduction:

- a. Project overview and objectives.
 - b. Stakeholders involved in the project.
 - c. The purpose of the system.
1. **Functional Requirements:** Detailed description of each function the system must perform. Example: "The system shall allow users to reset their passwords using their registered email address."
 2. **Non-functional Requirements:** Performance, security, usability, and other constraints. Example: "The system shall encrypt user data using AES-256 encryption."
 3. **Use Cases or User Stories:** Use cases are descriptions of how users will interact with the system, often represented in a step-by-step format. User stories are short, simple descriptions of a feature from the perspective of the user. Example: "As a user, I want to be able to view my order history to track my purchases."
 4. **Assumptions and Dependencies:** Any assumptions made during the analysis and any external dependencies or integrations (e.g., with other systems or services).
 5. **Acceptance Criteria:** Clear conditions that must be met for the system to be accepted by stakeholders. These criteria help testers to validate the requirements during the testing phase. Example: "A user must be able to complete the purchase process within 2 minutes."

C. Importance of Requirements Analysis and Specification

1. **Prevents Miscommunication:** Ensures that all stakeholders, including developers, testers, and clients, have a shared understanding of what the system should do.
2. **Reduces Cost and Time:** Errors or ambiguities in requirements, if discovered later in development, can be costly to fix. Proper analysis and specification help catch potential issues early.
3. **Serves as a Reference:** The requirements specification serves as a reference throughout the development cycle, helping designers, developers, and testers to ensure that the system aligns with stakeholder expectations.
4. **Improves Project Management:** Clear requirements help project managers allocate resources, estimate timelines, and manage risks more effectively.
5. **Facilitates Testing and Validation:** Well-documented requirements are essential for creating test cases to validate that the system functions correctly. It ensures the software meets the agreed-upon functionality and performance standards.

D. Common Challenges in Requirements Analysis and Specification

1. **Changing Requirements:** Stakeholders may change their expectations as the project progresses. Managing evolving requirements is crucial, especially in long projects.
2. **Ambiguity and Vagueness:** Poorly defined requirements can lead to different interpretations by developers and testers, resulting in a system that does not meet user needs.
3. **Stakeholder Conflicts:** Different stakeholders may have conflicting requirements, which need to be resolved during analysis.
4. **Unclear Non-functional Requirements:** Sometimes, non-functional requirements like performance, security, or scalability are not clearly defined, which can lead to poor system performance or user dissatisfaction.

Metrics for Software Project Size Estimation

Estimating the size of a software project is a critical step in project planning, as it affects effort, cost, duration, and resource allocation. Various metrics are used to estimate the size of a software project. The most common size estimation metrics include:

- 1) LOC
- 2) FP Metrics

Lines of Code (LOC): The total number of lines in the source code, including executable statements and comments.

- 1) **Usage:** A basic and traditional metric, used to measure the size of the software by counting the lines of code.
- 2) **Advantages:**
 - a. Simple and easy to understand.
 - b. Provides direct correlation to effort and time needed for development.
- 3) **Disadvantages:**
 - a. Language-dependent: Different languages require different amounts of code to solve the same problem.
 - b. No focus on functionality: Doesn't capture the actual business functionality provided to the user.
 - c. Penalizes code quality: May encourage longer, less efficient code.

Function Point Metrics

Function Point (FP) is a widely used software measurement technique to estimate the size and complexity of a software project. Unlike the traditional Lines of Code (LOC) metric, which focuses on the amount of code, Function Point measures the functionality delivered to the user, providing a more user-centric and technology-independent way of estimating project effort.

A. Components of Function Points

Function Points are calculated by evaluating different aspects of a software system, categorized into five basic components:

1. **External Inputs (EI):** Inputs from users or external systems. Each input adds value by modifying data or controlling system behaviour. Example: Login Form, Registration Page, Data Upload, Search Query.
2. **External Outputs (EO):** Data or information sent to external systems or users. Includes outputs that do not update data but are generated from internal logic. Example: Report Generation, Invoice Print, Confirmation Message, Search Results Display.
3. **External Inquiries (EQ):** Interactive inputs and outputs that retrieve and display data (e.g., search results). They typically do not modify data but allow data access and presentation. Example: Search Database, Retrieve User Info, Product Availability Check, Order Status Query.
4. **Internal Logical Files (ILF):** Logical groups of data maintained and processed by the system (e.g., database tables). Files that are internal to the application and contain persistent data. Example: Customer Database, Product Catalogue, Order Records, Employee Data.
5. **External Interface Files (EIF):** Data managed by external applications, but accessed and used by the system (e.g., external databases). These files are used by the system but not maintained by it. Example: Third-party API, External Inventory System, Payment Gateway, Partner Data Feeds.

B. Steps to Calculate Function Points

1. **Determine the Unadjusted Function Points (UFP):**
 - a. Assign weights (low, average, or high complexity) to each of the five components (EI, EO, EQ, ILF, EIF).
 - b. Each component is given a weight based on its complexity, such as:
 - i. External Inputs (EI): 3 (low), 4 (medium), 6 (high)
 - ii. External Outputs (EO): 4 (low), 5 (medium), 7 (high)
 - iii. External Inquiries (EQ): 3 (low), 4 (medium), 6 (high)
 - iv. Internal Logical Files (ILF): 7 (low), 10 (medium), 15 (high)
 - v. External Interface Files (EIF): 5 (low), 7 (medium), 10 (high)
 - c. Multiply the number of components in each category by their assigned weight, then sum them up to calculate the **Unadjusted Function Points (UFP)**.
2. **Adjust for Complexity:**
 - a. After calculating the UFP, adjust the count based on 14 general system characteristics (e.g., data communications, performance, transaction rates, security).
 - b. These characteristics are rated on a scale of 0 to 5, where 0 means no influence and 5 means strong influence.
- c. Compute the **Value Adjustment Factor (VAF)/Product Complexity adjustment (PC)**:

$$\text{VAF} = 0.65 + (0.01 \times \text{Sum of Influence Factors})$$

3. Calculate Adjusted Function Points:

$$FP = UFP \times VAF$$

C. Formula Summary

1. Unadjusted Function Points (UFP):

$$UFP = \left(\sum (\text{weight for each component} \times \text{count of each component}) \right)$$

2. Value Adjustment Factor (VAF):

$$VAF = 0.65 + (0.01 \times \text{Sum of Influence Factors})$$

3. Adjusted Function Points (FP):

$$FP = UFP \times VAF$$

D. Advantages of Function Points

- **Technology Independent:** Focuses on functionality rather than programming language or technology used.
- **User-Centric:** Measures the value delivered to the user rather than focusing purely on the technical aspects.
- **Useful in Early Estimation:** Can be applied early in the development process, when LOC estimates may not be available.

E. Disadvantages of Function Points

- **Subjectivity in Complexity Rating:** Assigning complexity levels and influence factors can be subjective.
- **Requires Expertise:** Estimating function points accurately requires a good understanding of the software's functionality.

Function Point Worksheet

Measurement parameter	Count		Weighting Factor			Choice		
			simple	average	complex			
# of user inputs	11	X	3	4	6	4	=	44
# of user outputs	13	X	4	5	7	5	=	65
# of user inquiries	3	X	3	4	6	4	=	12
# of files	1	X	7	10	15	10	=	10
# of external interfaces	0	X	5	7	10	7	=	0
Count-total (UFP)=								131

Rate each factor on a scale of 0 to 5:

0 - No Influence	1 - Incidental	2 - Moderate
3 - Average	4 - Significant	5 - Essential

1. Does the system require reliable backup and recovery?	5	
2. Are data communications required?	4	
3. Are there distributed processing functions?	1	
4. Is performance critical?	3	
5. Will the system run in an existing, heavily utilized operational environment?	4	
6. Does the system require on-line data entry?	1	
7. Does the on-line data entry require the input transaction to be built over multiple screens or operations?	2	
8. Are the master files updated on-line?	3	
9. Are the inputs, outputs, files, or inquiries complex?	2	
10. Is the internal processing complex?	4	
11. Is the code designed to be reusable?	1	
12. Are conversion and installation included in the design?	3	
13. Is the system designed for multiple installations in different organizations?	2	
14. Is the application designed to facilitate change and ease of use by the user?	3	
Total Complexity Adjustment Value =		38

Product Complexity Adjustment (PC) = $[.65 + .01 \cdot \text{CAV}] = 1.03$

Total Adjusted Function Point (FP) = $\text{UFP} \cdot \text{PC} = 134.93$

Language Factor (LF) = 60

Source Lines of Code (SLOC) = $\text{FP} \cdot \text{LF} = 8095.8$

BASIC COCOMO MODEL (ORGANIC)

LOC	8095.8			
KLOC	8.0958	Cost per month (in Rs)	50000	
		a	2.4	
		b	1.05	
		c	0.38	

EFFORT (in PM) 21.5716816

T_{dev} (in M) 8.03185629

STAFF SIZE (in P) 3

Cost to develop (in Rs) 401592.814

COCOMO Model (Constructive Cost Model)

The **COCOMO Model** is a widely-used algorithmic software cost estimation model developed by **Barry Boehm** in 1981. It is used to estimate the effort, cost, and schedule required to develop software projects based on their size (in lines of code or function points) and various other project characteristics. Key Components of the COCOMO Model:

1. **Effort (Person-Months):** Amount of effort required to complete the project.
2. **Development Time:** The total time needed to develop the project.
3. **Software Size:** The size of the software, typically measured in **Kilo Lines of Code (KLOC)**.
4. **Effort Multipliers and Cost Drivers:** Factors that influence the project cost and effort, like team experience, tools used, etc.

A. Types of Software Products

1. **Organic:** Simple, small-sized projects with well-understood domains, limited innovation, and experienced teams. Stable requirements, low complexity, small team size, familiar development environment. **Examples:** Accounting software, inventory management systems.

2. **Semidetached:** Medium-sized projects with a mix of well-known and unfamiliar technologies and domains. Medium complexity, moderate experience in the domain, larger team size than organic projects. **Examples:** Applications with both custom and standard modules (e.g., payroll systems with additional features).
3. **Embedded:** Complex, real-time, or safety-critical software that requires close hardware-software interaction. High complexity, stringent performance, reliability, and security requirements **Examples:** Operating systems, real-time systems, defense or aerospace systems.

Product Type	Project Size	Complexity	Team Size	Familiarity	Deadline of project	Example
Organic	Small (2-50 KLOC)	Low	Small	Well-known domains	Not tight	Accounting software
Semi detached	Medium (50-300 KLOC)	Medium	Medium	Mix of familiar or unfamiliar	medium	Payroll system
Embedded	Large (300+ KLOC)	High	Large	Unfamiliar, custom	tight	Real-time operating systems

B. COCOMO Model Levels

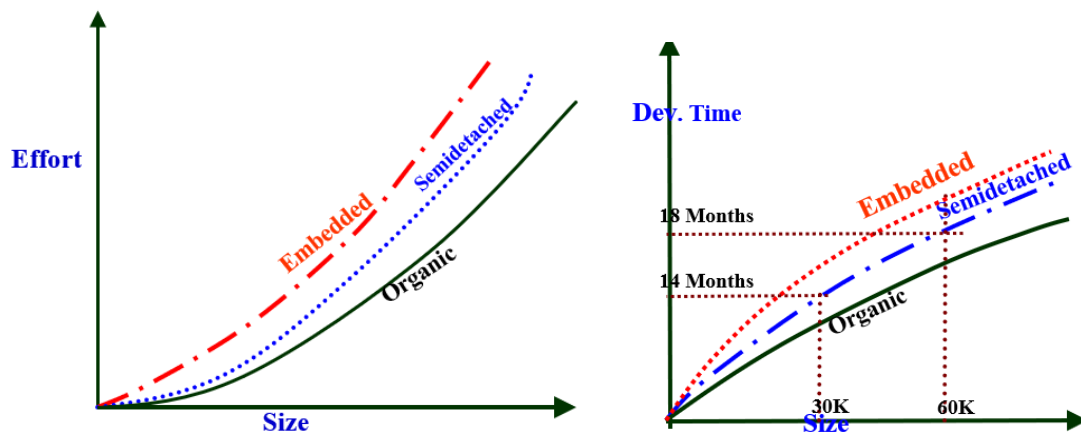
1. **Basic COCOMO:** The simplest form of the COCOMO model, where effort and cost estimation are based solely on the size of the software (in KLOC).

- **Formula:** Where **a** and **b** are constants that vary by software type (organic, semidetached, embedded).

$$Effort = a \times (KLOC)^b \quad \text{Time (TDEV)} = 2.5 \times Effort^c$$

- **Usage:** Good for early, rough estimates when limited information is available about the project.
- **Shortcomings:** Does not account for different factors like team experience, development environment, or complexity. It offers only a basic estimation.

Software project	a	b	c
Organic	2.4	1.05	0.38
Semi – detached	3.0	1.12	0.35
Embedded	3.6	1.2	0.32



For example, a 60 KLOC program can be developed in approximately 18 months, regardless of whether it is organic, semi-detached, or embedded. There is more scope for parallel activities for system and application programs, than utility programs.

- Effort = $2.4 \times (60)^{1.05}$ PM. = 176.713 PM. ➤ $T_{dev} = 2.5 \times (176.713)^{0.38}$ Months. = 17.8 Months.
- Effort = $3.0 \times (60)^{1.12}$ PM. = 294.205 PM. ➤ $T_{dev} = 2.5 \times (294.205)^{0.35}$ Months. = 18.27 Months.
- Effort = $3.6 \times (60)^{1.20}$ PM. = 489.873 PM. ➤ $T_{dev} = 2.5 \times (489.873)^{0.32}$ Months. = 18.14 Months.

2. **Intermediate COCOMO:** Builds on Basic COCOMO by including **cost drivers** or factors that influence development effort, such as product attributes, hardware constraints, personnel capabilities, and project characteristics. Provides a more detailed and accurate estimate by factoring in 15 cost drivers.

- **Formula:** Where **EAF** (Effort Adjustment Factor) is derived from a set of cost drivers (like reliability, database size, and team expertise).

$$Effort = a \times (KLOC)^b \times EAF \quad \text{Time (TDEV)} = 2.5 \times Effort^c$$

Software project	a	b	c
Organic	3.2	1.05	0.38
Semi – detached	3.0	1.12	0.35
Embedded	2.8	1.2	0.32

- The size of a software product has been estimated to be 200 KLOC. Cost drivers are as follows:

- Low Reliability 0.88
- High Product Complexity 1.15
- Low Application Experience 1.13
- High Programming Language Experience 0.95

- $EAF = 0.88 * 1.15 * 1.13 * 0.95 = 1.086$

- Semi-Detached:

- $Effort|_{corrected} = Effort * EAF = 3.0 * (200)^{1.12} * 1.086 = 1230.56 = 1231$ PM

- $Tdev|_{corrected} = 2.5 * (Effort|_{corrected})^c = 2.5 * (1231)^{0.35} = 30.16$ months = 30 months

3. **Detailed/Complete COCOMO:** The most comprehensive model, where the software project is broken down into components, and separate COCOMO estimates are applied to each module. The detailed model includes phase-sensitive effort multipliers for each stage of the project. **Usage:** Offers the most precise and refined cost estimation for large-scale projects by considering various phases (design, coding, testing).

Aspect	Basic COCOMO	Intermediate COCOMO	Complete/Advanced COCOMO
Definition	The simplest version of the COCOMO model, estimating effort based on the size of the software (KLOC).	Extends Basic COCOMO by incorporating cost drivers (effort multipliers) related to the project and development environment.	Breaks down the software project into components and applies separate cost drivers for each phase of development.

Main Inputs	KLOC (Lines of Code)	KLOC and 15 cost drivers (e.g., complexity, personnel, tools)	KLOC, cost drivers, and detailed phase breakdown of the project.
Cost Drivers	None	15 cost drivers such as product reliability, experience, team cohesion, and tools.	Phase-specific cost drivers for each stage of the development process.
Accuracy	Low to Moderate	Moderate to High, depending on the accuracy of cost driver ratings.	Highest accuracy due to detailed phase breakdown and tailored cost drivers.
Use Case	Suitable for small, simple projects with well-defined requirements where rough early estimates are needed.	Appropriate for mid-sized projects with more variability in complexity, team expertise, and development conditions.	Best for large-scale, complex projects requiring precise and detailed estimations at each phase.
Complexity	Low – simple and easy to apply.	Moderate – requires more inputs and understanding of cost drivers.	High – requires detailed project breakdown and precise phase estimation.
Flexibility	Rigid – limited to size (KLOC) and cannot accommodate project-specific variations.	More flexible than Basic, accounts for different aspects of project complexity, team skills, and environment.	Very flexible – can adapt to various project phases, complexities, and cost drivers for better precision.
Advantages	Quick and easy to use for early estimation in small projects.	More accurate due to inclusion of cost drivers, adaptable for different project types.	Provides the most accurate estimates, especially for complex, phase-sensitive projects.
Disadvantages	Doesn't consider team experience, project risks, or other influencing factors.	Relies on subjective rating of cost drivers, and is harder to apply than Basic COCOMO.	Highly complex, requires detailed inputs, and is time-consuming to implement.
Effort Drivers	Only considers KLOC (lines of code).	Considers multiple factors affecting project effort like product, personnel, and environmental factors.	Accounts for all factors considered in Intermediate COCOMO plus phase-specific attributes.
Best For	Small projects with well-understood domains.	Medium-sized projects with a mix of known and unknown factors.	Large-scale projects, or projects with complex requirements and significant uncertainty.

C. Shortcomings of Basic and Intermediate COCOMO

1. Basic COCOMO:

- **Lack of Flexibility:** Only considers software size (KLOC) without accounting for project complexity, team expertise, or other factors.
- **Inaccurate for Large Projects:** For complex or larger projects, the estimation can be inaccurate as it doesn't consider important cost drivers.

2. **Intermediate COCOMO:**

- **Cost Driver Subjectivity:** The model introduces cost drivers, but evaluating them accurately can be subjective, leading to variability in estimates.
- **Static Estimates:** Though more refined than Basic COCOMO, Intermediate COCOMO still doesn't handle dynamic changes during the project well (e.g., requirement changes).

D. Advantages:

1. **Simplicity:** Easy to understand and apply, especially in the Basic COCOMO model.
2. **Well-Defined:** Provides a structured framework to estimate cost and effort, making it useful for early project planning.
3. **Adaptability:** The Intermediate and Complete COCOMO models account for various cost drivers, making estimates more adaptable to different project types.
4. **Historical Data Usage:** COCOMO leverages historical data to improve the accuracy of cost estimation.

E. Disadvantages:

1. **Dependency on Historical Data:** Requires reliable historical data, which may not always be available or relevant for new types of projects.
2. **Limited Flexibility in Basic COCOMO:** Basic COCOMO doesn't account for cost drivers like team experience, hardware limitations, or complexity, making it unsuitable for complex projects.
3. **Complexity in Detailed COCOMO:** While offering more accurate estimates, Complete COCOMO can be very complex to apply, requiring significant effort and input data.
4. **Assumes KLOC-Based Estimation:** The model's reliance on KLOC for size measurement is outdated for modern software development practices that focus on reusable code, agile processes, and more abstract designs like microservices.

COCOMO II (Constructive Cost Model II)

COCOMO II is an advanced version of the original **COCOMO (Constructive Cost Model)**, designed to estimate the cost, effort, and schedule for software development projects. It adapts to the modern software development environment, accounting for different project types and lifecycle models. COCOMO II is more flexible and better suited for contemporary software projects that may involve prototyping, reuse, and rapid application development (RAD).

COCOMO II has three sub-models:

1. Application Composition Model:

- Used for early design stages of a project where prototypes or RAD techniques are applied.
- Focuses on projects developed using reusable components.

2. Early Design Model:

- Used when a rough design of the software system is available.
- Provides a quick, high-level estimate for effort and schedule based on early project data.

3. Post-Architecture Model:

- Applied when the project design is stable, and the full architectural details are available.
- Provides the most detailed estimates for effort, cost, and time, considering factors like scalability, complexity, and developer experience.

COCOMO II Estimation Factors

- **Scale factors:** Reflect the software development process and its scalability (e.g., team cohesion, process maturity).
- **Effort multipliers:** These adjust the effort based on specific project attributes such as product reliability, platform constraints, and personnel capability.

Cost-Benefit Analysis (CBA)

Cost-Benefit Analysis (CBA) is a financial assessment tool used to evaluate the economic feasibility of a project by comparing its costs with the anticipated benefits. **Key Steps in Cost-Benefit Analysis:**

1. **Identify Costs:**
 - **Direct Costs:** Hardware, software, developer salaries, etc.
 - **Indirect Costs:** Maintenance, training, support, etc.
 - **Opportunity Costs:** Benefits foregone by not choosing alternative projects.
2. **Identify Benefits:**
 - **Tangible Benefits:** Improved productivity, reduced errors, increased sales, etc.
 - **Intangible Benefits:** Enhanced customer satisfaction, better decision-making, competitive advantage.
3. **Quantify Costs and Benefits:** Assign monetary values to both costs and benefits. Some benefits may be hard to quantify but can still be expressed as estimates.
4. **Time Horizon:** Consider the time over which the costs and benefits will accrue (e.g., 3, 5, or 10 years).
5. **Discount Rate:** Apply a discount rate to future costs and benefits to account for the time value of money. This converts future benefits into present value terms.
6. **Perform Net Benefit Calculation:** $\text{Net Benefit} = \text{Total Benefits} - \text{Total Costs}$. If the net benefit is positive, the project is considered economically viable.
7. **Sensitivity Analysis:** Assess how sensitive the outcomes are to changes in key variables, such as cost estimates, benefit projections, or discount rates.

Advantages of CBA:

- **Simple and Intuitive:** Provides a straightforward method for evaluating the financial viability of projects.
- **Data-Driven:** Helps in making informed decisions based on quantitative financial data.
- **Broad Applicability:** Can be used across various types of projects, from software development to infrastructure investments.

Disadvantages of CBA:

- **Difficult to Quantify Intangible Benefits:** Some benefits, like user satisfaction or brand loyalty, are hard to express in monetary terms.
- **Uncertainty in Estimates:** Costs and benefits are often based on assumptions, which can lead to inaccuracies.
- **Short-Term Focus:** CBA may favour projects with short-term returns over those with long-term strategic importance.

Characteristics of a Good SRS Document

A **Software Requirements Specification (SRS)** document outlines the functional and non-functional requirements of a software system. It serves as a communication tool between stakeholders, developers, and testers. **Key Characteristics of a Good SRS Document**

1. **Correctness:** The document accurately reflects the system's requirements, capturing everything the software needs to accomplish.
2. **Completeness:** All functionalities, constraints, and specifications are clearly and fully defined. There should be no missing information that could lead to ambiguity during development.
3. **Unambiguity:** Each requirement must be clearly and precisely described to avoid misinterpretation. Terms should be well-defined, and technical jargon should be used consistently.
4. **Consistency:** The document should not have conflicting requirements or descriptions. All sections must be aligned and logically coherent.
5. **Verifiability:** Each requirement should be testable and measurable, ensuring that it can be verified through testing or inspection once the software is developed.
6. **Traceability:** The origin of each requirement should be clear, and the document should provide links between each requirement and its corresponding system component or business need.
7. **Modifiability:** The document should be structured in a way that allows easy updates as requirements evolve. Clear organization and formatting contribute to this.
8. **Feasibility:** The requirements described must be realistic and achievable within the constraints of technology, time, and budget.
9. **Prioritization:** Requirements should be ranked by their importance to the project's success, helping guide developers in delivering the most critical functionality first.

Advantages & disadvantages of the SDLC models

SDLC Model	Advantages	Disadvantages
Classical Waterfall Model	1. Simple and easy to understand 2. Well-structured stages 3. Clear documentation with well-defined input and output. 4. Well suited for low and fixed price contracts.	1. Rigid, no revisions 2. No changes once the stage is completed 3. High development time. 4. Not ideal for the complex and uncertain projects 5. High risk and uncertainty.

Iterative Waterfall Model	1. Allows feedback at each stage 2. Early detection of issues 3. Structured and disciplined approach. 4. Better risk management. 5. More reliable and higher quality of final product.	1. More complex than classical 2. Limited no of iterations 3. Large development time. 4. Risk of over iteration. 5. Managing iterations and feedback is complex.
Prototyping Model	1. Early detection of issues 2. Provides working prototype early 3. Better requirement understanding. 4. Risk reduced.	1. Prototype may lack structure 2. Increases cost and time 3. Highly complex. 4. Due to quick builds prototype lack perfection structure and scalability.
Evolutionary Model	1. Evolving requirements 2. Higher flexibility. 3. Improved user involvement. 4. Improved risk. 5. Useful for larger size projects	1. Difficult to manage constant changes 2. Time of development is large.
Spiral Model	3. Focuses on risk management 4. Combines iterative and feedback development 5. Early problem identification.	1. Complex and costly 2. Unsuitable for small projects 3. Over emphasis of risk. 4. May suffer due to need of higher expertise.
RAD Model	1. Quick delivery 2. Encourages user feedback 3. Reduce the risk.	1. Needs skilled developers 2. Not ideal for large projects 3. Quality concerns due to quick development.
V Model	1. Emphasizes verification and validation 2. Suitable for small projects 3. Easy to understand and use.	1. Rigid, handles changes poorly 2. Testing starts after development 3. Risk of unclear requirements. 4. Not suitable for complex and long-term projects.
Agile Model	1. Highly flexible 2. Continuous feedback and collaboration	1. Requires strong team collaboration 2. Less predictable outcomes